

PRÉPARATION À L'ORAL · SOUTENANCE TP

GitOps & ArgoCD

Déploiement Continu sur Kubernetes avec une approche Pull-based

ÉTUDIANT

Mathis Lefebvre

CLASSE

5ESGI SRC

PROJET

DevHub Campus

DURÉE ESTIMÉE

15 min + 5 min démo

● ArgoCD v3.4

● Kubernetes / Kind

● Helm & GitOps

Sommaire & Structure

1

Diapositive 01

Contexte & Objectifs du TP

2

Diapositive 02

Les Concepts Clés du GitOps

3

Diapositive 03

Architecture & Infrastructure

4

Diapositive 04

Docker & Helm — Standardisation

5

Diapositive 05

Bootstrap App of Apps

6

Diapositive 06

Environnements Éphémères
(ApplicationSets)

7

Diapositive 07

Sécurité & RBAC

8

Diapositive 08

Observabilité & Alerting

9

Diapositive 09

Limites & Risques en Production

10

Diapositive 10

Bilan & Retours d'Expérience

Démo Interface ArgoCD



Démo — Phase 1

Préparation & Lancement du Cluster



Démo — Phase 2

Self-Healing en Direct



Démo — Phase 3

Démonstration du RBAC



Démo — Phase 4

Preview Environments

🎯 Objectif du TP

Mettre en place une plateforme **GitOps** **complète** pour le projet *DevHub Campus* — un annuaire d'étudiants sur Kubernetes géré 100% depuis Git, sans intervention manuelle sur le cluster.

📦 Ce qui est déployé

- **annuaire** — Front Node.js (répertoire étudiant)
- **planning** — API Python (gestion des plannings)
- **notif** — Service Go (notifications push)
- **ArgoCD** — moteur GitOps (orchestrateur central)

9+

ÉTAPES RÉALISÉES

3

SERVICES CONTENEURISÉS

100%

GÉRÉ PAR GIT

🔧 Environnement technique

kubectl v1.34

Helm v3.21

ArgoCD CLI v3.4

Kind v0.24

Docker Engine v27

Cluster Kind 2 nœuds (*control-plane* + *worker*), Ingress-NGINX exposant les services localement via le fichier `hosts` Windows.

🎤 SCRIPT ORAL

"Ce TP m'a permis de mettre en place une chaîne de déploiement continu complète basée sur ArgoCD. L'idée centrale est simple : tout ce qui tourne sur le cluster Kubernetes doit être décrit dans un dépôt Git et jamais modifié manuellement. On appelle ça le GitOps. Je vais vous présenter comment j'ai architecturé ça, ce que ça apporte concrètement en termes de sécurité et d'automatisation, et je vous ferai une démonstration live sur l'interface ArgoCD."

❌ Mode Push (CI classique)

- Le runner CI (GitHub Actions) possède les clés admin du cluster
- Risque majeur : si la CI est compromise → le cluster entier l'est aussi
- Pas de détection des modifications manuelles
- Pas de réconciliation automatique

✅ Mode Pull (GitOps)

- ArgoCD tourne *dans* le cluster et lit le dépôt Git
- Aucune clé admin n'est exposée à l'extérieur
- Toute modification hors Git est corrigée automatiquement
- Git = seule source de vérité

📖 Glossaire fondamental

- **AppProject** : barrières RBAC (quels dépôts, quels namespaces)
- **Application** : lien Git → Namespace K8s
- **ApplicationSet** : générateur automatique d'Applications
- **Boucle de réconciliation** : comparaison Git/Cluster en continu
- **Self-Healing** : correction automatique des modifications manuelles
- **Pruning** : suppression auto des ressources retirées de Git
- **Drift** : décalage entre Git et l'état réel du cluster
- **Sync Wave** : ordre de déploiement (DB avant API avant Front)

⚙️ Les 4 Piliers du GitOps

① Déclaratif (YAML/Helm)

② Git = Source de vérité

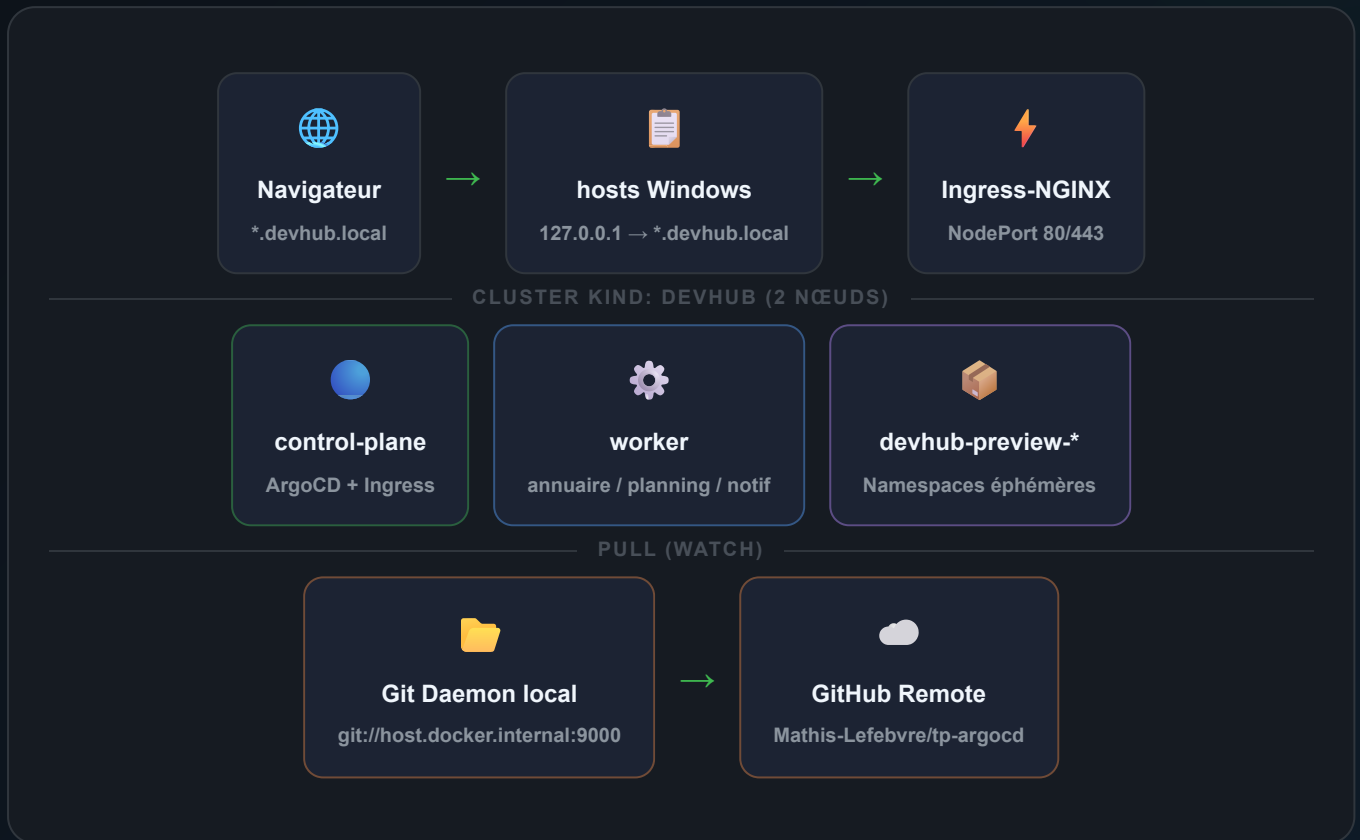
③ Application automatique

④ Réconciliation continue



SCRIPT ORAL

"La différence fondamentale entre le GitOps et un pipeline CI classique, c'est le sens du flux. En CI classique, votre pipeline va pousser des changements vers le cluster avec des droits admin. Avec ArgoCD, on inverse : c'est l'agent qui tire depuis Git depuis l'intérieur du cluster. Si quelqu'un fait un `kubectl edit`



🌐 Namespaces

- `argocd` — ArgoCD lui-même
- `devhub-dev` — Environnement de dev
- `devhub-preview-*` — Previews éphémères par branche

🔗 URLs locales

- `argocd.devhub.local` — Interface ArgoCD
- `annuaire.devhub.local` — Frontend
- `planning.devhub.local` — API
- `annuaire-feature-demo-prof.devhub.local`

🎙️ SCRIPT ORAL

"J'ai monté l'infrastructure en local avec Kind, qui crée un vrai cluster Kubernetes multi-nœuds dans Docker. Le cluster expose ses services via Ingress-NGINX et j'ai configuré le fichier hosts Windows pour résoudre les domaines en local. ArgoCD tire les manifestes Kubernetes depuis un serveur Git local — ce qui reproduit

exactement le comportement d'un environnement de production. La source de vérité est le dépôt GitHub Mathis-Lefebvre/tp-argocd."

Docker & Helm — Conteneurisation & Packaging

🕒 ~2
min

📦 annuaire

Node.js 20

- Multi-stage build
- Image runtime `node:20-alpine`
- Taille : ~130 Mo
- Utilisateur `node` (non-root)
- Port `8080`

🔄 planning

Python 3.12

- Multi-stage build + venv
- Image `python:3.12-slim`
- Taille : ~155 Mo
- Utilisateur `appuser` (non-root)
- Port `8080`

⚡ notif

Go statique

- Multi-stage build Go
- Image `distroless/static`
- Taille : ~22 Mo
- ★ Minimal
- Utilisateur `nonroot`
- Pas de shell disponible

🔒 Durcissement Helm / Kubernetes (SecurityContext)

- `runAsNonRoot: true`
- `allowPrivilegeEscalation: false`
- `readOnlyRootFilesystem: true`
- `livenessProbe` & `readinessProbe` sur `/healthz`
- Requests & Limits CPU/Mémoire définies
- Labels OCI standards dans les helpers

🎙️ SCRIPT ORAL

"Avant de parler d'ArgoCD, j'ai travaillé sur la qualité de ce qu'on déploie. Les trois services sont conteneurisés avec des Dockerfiles multi-stages : on sépare la phase de build de l'image de production. Résultat : le service de notification en Go pèse seulement 22 Mo parce qu'il utilise une image distroless, sans shell ni OS. Niveau sécurité Kubernetes, j'ai appliqué le principe du moindre privilège sur tous les conteneurs — ils tournent en utilisateur non-root, avec un système de fichiers en lecture seule."

Pattern App of Apps & Root Application

🕒 ~1.5
min

🌳 Pourquoi App of Apps ?

Sans App of Apps (`kubectl apply -f`)

- ✗ Ne supprime rien si on retire un fichier YAML
- ✗ N'empêche pas les modifications manuelles
- ✗ Pas de visibilité centralisée

Avec App of Apps + ArgoCD

- ✓ Pruning auto (suppression si retiré de Git)
- ✓ Self-healing (correction des drifts)
- ✓ Arbre de dépendances dans l'UI ArgoCD

📁 Structure du dépôt (vue logique)

📁 ARBORESCENCE DEVHUB-CAMPUS/

```
platform/
├─ bootstrap/
│   └─ root-app.yaml      # 👑 Application racine (bootstrapper)
├─ apps/
│   └─ dev/               # Applications ArgoCD par environnement
│       ├── annuaire-dev.yaml
│       ├── planning-dev.yaml
│       └─ notif-dev.yaml
│   └─ sets/
│       └─ preview-appset.yaml # 📄 ApplicationSet previews
├─ projects/
│   └─ devhub.yaml        # AppProject (périmètre RBAC)
└─ argocd/
    └─ values.yaml        # Config ArgoCD (RBAC, notifs...)

services/
├─ annuaire/              # Chart Helm + Dockerfile
├─ planning/
└─ notif/
```



"J'ai utilisé le pattern App of Apps pour bootstrap toute la plateforme avec une seule commande. On a une application racine dans ArgoCD qui surveille le dossier `pLatform/apps/` dans Git. Dès qu'on ajoute un fichier YAML d'Application à ce dossier et qu'on commit, ArgoCD le détecte et crée automatiquement la ressource sur le cluster. De même, si on supprime ce fichier, l'application et toutes ses ressources Kubernetes sont pruned — proprement supprimées."

ApplicationSets & Preview Environments

~2
min

Flux de création automatique

- ① Push branche `feature-X` →
- ② ApplicationSet détecte la branche →
- ③ Namespace créé: `devhub-preview-feature-X` →
- ④ Ingress unique: `annuaire-feature-X.devhub.local` →
- ⑤ Suppression branche → Pruning auto

Générateur List (maquette locale)

Pour simuler le comportement sans token GitHub API, on utilise un *List Generator* avec la branche `feature-demo-prof` hardcodée. En production, on utiliserait le *Pull Request Generator* pour un déclenchement automatique sur ouverture de PR.

Variables de template

- `{{`{{branch}}`}}` → `feature-demo-prof`
- `{{`{{branch_slug}}`}}` → `feature-demo-prof`
- Namespace: `devhub-preview-{{`{{branch_slug}}`}}`
- Host: `annuaire-{{`{{branch_slug}}`}}.devhub.local`



Avantage majeur : chaque développeur dispose de son propre environnement de test isolé, créé en quelques secondes depuis un simple push de branche. Plus besoin de se disputer un environnement de staging partagé.



SCRIPT ORAL

"Les ApplicationSets permettent de générer des Applications ArgoCD dynamiquement. Dans mon cas, j'ai créé un ApplicationSet qui surveille une liste de branches. Quand une nouvelle branche apparaît, ArgoCD crée automatiquement un namespace isolé, déploie l'annuaire dedans, et configure un Ingress avec un nom de domaine unique pour cette branche. Quand la branche est supprimée, tout est nettoyé automatiquement

— c'est le pruning. En production, ce générateur se connecte directement à l'API GitHub pour réagir aux Pull Requests."

RBAC, AppProject & Isolation Multi-tenant

~2
min

Compte admin

- Login: `admin`
- Accès complet à toutes les applications
- Peut modifier les projets et la config ArgoCD
- Rôle: `role:admin`

Compte developer (restreint)

- Login: `developer`
- Lecture seule sur toutes les apps
- Sync autorisé *uniquement* sur `annuaire-*`
- **Bloqué** sur `planning-dev` et `notif-dev`

Règles RBAC configurées (argocd/values.yaml)

🔒 POLITIQUE RBAC CASBIN

```
# Droits du groupe developer
p, role:developer, applications, get,    devhub/*, allow
p, role:developer, applications, sync,   devhub/annuaire-*, allow
p, role:developer, applications, sync,   devhub/planning-*, deny
p, role:developer, applications, sync,   devhub/notif-*, deny

# Résultats validés en CLI
argocd app sync annuaire-dev → OK ✅
argocd app sync planning-dev → permission denied ❌
```

🛡️ AppProject devhub

- Sources autorisées : uniquement `git://host.docker.internal:9000/`
- Destinations autorisées : cluster local + namespaces `devhub-*`
- Ressources bloquées : `Namespace` (création non autorisée)



"J'ai configuré deux niveaux de contrôle d'accès. D'abord l'AppProject qui définit le périmètre : quels dépôts Git sont autorisés, sur quels namespaces peut-on déployer. Ensuite le RBAC : j'ai créé un compte developer qui peut voir toutes les apps mais ne peut synchroniser que l'annuaire. Si le développeur essaie de syncer le service planning, ArgoCD lui retourne un permission denied. Je vous montrerai ça en démo."

Métriques Prometheus & Alertes Webhook

 ~1.5
min

argocd_app_info

Suivi en temps réel de l'état de santé (*Healthy/Degraded*) et de synchronisation de chaque application. Permet les dashboards Grafana.



argocd_app_reconcile_duration_seconds

Mesure le temps de réconciliation Git↔Cluster. Permet de détecter les surcharges du contrôleur.



argocd_app_s

Comptage des synchronisations. détecter les boucles infinies (signe de de configuration).



Notifications Webhook configurées

Le module de notifications natif ArgoCD est activé. En cas d'échec de synchronisation (`on-sync-failed`), un JSON est envoyé vers `webhook.site` contenant :

`Nom de l'application``Commit cible (SHA)``Détail de l'erreur K8s``Timestamp`

En production, ce webhook serait connecté à **Slack, PagerDuty ou Teams** pour alerter l'équipe d'astreinte immédiatement, sans nécessiter de polling manuel de l'interface.



SCRIPT ORAL

"ArgoCD expose nativement des métriques Prometheus. J'ai identifié les trois métriques clés pour une supervision en production. J'ai aussi configuré le système de notifications intégré : dès qu'un déploiement échoue, un webhook est envoyé automatiquement. En entreprise, on brancherait ça directement sur Slack ou PagerDuty pour que l'équipe d'astreinte soit alertée sans avoir à surveiller manuellement l'UI."

Limites d'ArgoCD & Risques en Production

 ~2
min

⚠ Ce qu'ArgoCD ne gère pas nativement

- **Déploiements canary / blue-green** → nécessite Argo Rollouts ou Flagger
- **Gestion des secrets Git** → il ne faut jamais stocker des mots de passe en clair dans Git
- **Policy-as-Code** → Kyverno ou OPA Gatekeeper pour bloquer les déploiements non-conformes
- **Tests de smoke post-déploiement** → nécessite un pipeline CI complémentaire

🔒 Risque #1 : Secrets dans Git

Ne jamais stocker de mots de passe en clair.

Solution :

Sealed Secrets ou

External Secrets Operator

+ HashiCorp Vault

🔒 Risque #2 : Accès Git = Accès Cluster

Si le dépôt est compromis, le cluster l'est aussi.

Solution :

Branch Protection +
revues obligatoires +
signature des commits
(GPG)

👑 Risque #3 : Privilèges ArgoCD

Le contrôleur peut avoir des droits `cluster-admin`.

Solution : Restreindre
via **AppProject** +
RBAC granulaire



SCRIPT ORAL

"ArgoCD est très puissant, mais il faut avoir conscience de ses limites. Le fait que Git soit la source de vérité signifie que si quelqu'un pousse du code malveillant sur votre branche principale, il contrôle votre cluster. D'où l'importance des branch protections et des revues de code. Concernant les secrets, on ne les stocke jamais en clair dans Git — on utilise des outils comme Sealed Secrets ou External Secrets Operator. Et pour les déploiements progressifs, ArgoCD seul ne suffit pas : il faut coupler ça avec Argo Rollouts."

Retours d'Expérience & Comparatif Outils

🕒 ~1.5 min

CRITÈRE	FLUX V2 (CNCF)	ARGOCD	HELM DIRECT (PUSH)
Modèle	Pull	Pull	Push
Interface Graphique	✗ Non (extensions tierces)	✓ UI native riche	✗ Non
Gestion du Drift	✓ Auto-correction	✓ Auto-correction	✗ Non gérée
Multi-tenancy / RBAC	Basé sur RBAC K8s	RBAC interne propre	RBAC du compte CI
Environnements éphémères	Kustomize + ImagePolicy	ApplicationSets natif	✗ Manuel
Courbe d'apprentissage	Moyenne	Faible (UI claire)	Très faible

✓ Ce que ce TP m'a apporté

- Maîtrise du modèle GitOps et de ses avantages vs CI classique
- Compréhension des concepts ArgoCD (App of Apps, ApplicationSets, RBAC)
- Pratique de la sécurité conteneur (non-root, distroless, securityContext)
- Gestion des environnements éphémères par branche
- Configuration de l'observabilité (métriques, notifications)
- Analyse critique des limites et risques en production



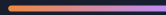
SCRIPT ORAL — CONCLUSION

"En conclusion, ArgoCD se démarque vraiment de Flux v2 par son interface graphique très riche qui facilite la prise en main et le débogage. Les ApplicationSets pour les environnements éphémères et le RBAC granulaire en font un choix solide pour une équipe avec plusieurs développeurs. Ce TP m'a permis de passer de la théorie du GitOps à une plateforme fonctionnelle en local, avec tous les concepts fondamentaux mis en pratique. Je suis maintenant prêt pour la démonstration live."

GUIDE DE DÉMONSTRATION LIVE

Démo ArgoCD UI

Script pas-à-pas pour la démonstration en direct · ~5 minutes



PHASE A

Préparation cluster

PHASE B

Self-Healing

PHASE C

RBAC developer

PHASE D

Preview Environments



À faire **AVANT** la présentation : Démarrer le cluster Kind et le serveur Git local. Ouvrir le navigateur sur `http://argocd.devhub.local`.

1

Vérifier que le cluster Kind tourne

```
kubectl get nodes  
# Attendu :  


| NAME                 | STATUS | ROLES         | AGE | VERSION |
|----------------------|--------|---------------|-----|---------|
| devhub-control-plane | Ready  | control-plane | Xh  | v1.29.x |
| devhub-worker        | Ready  | <none>        | Xh  | v1.29.x |


```

Si le cluster est arrêté : `make cluster-up` (⚠️ cela prend ~3 min)

2

Démarrer le serveur Git local

```
# Depuis le dossier devhub-campus  
git daemon --base-path=. --export-all --enable=receive-pack --reuseaddr --port=9000 --  
verbose &  
# Laisser le terminal ouvert ou envoyer en background
```

Ce daemon Git permet à ArgoCD (qui tourne dans le cluster) d'accéder au dépôt local.

3

Vérifier l'état de tous les pods ArgoCD

```
kubectl get pods -n argocd  
# Tous les pods doivent être Running (argocd-server, argocd-application-controller...)
```

4

Ouvrir l'interface ArgoCD

Navigateur → `http://argocd.devhub.local`

Login : `admin` / Mot de passe : `DevHubPassword123!`



À montrer : L'arbre des applications (root → annuaire-dev, planning-dev, notif-dev, preview-appset)

Démonstration du Self-Healing en Direct

 ~2
min

Objectif pédagogique : Montrer qu'ArgoCD corrige automatiquement toute modification manuelle sur le cluster, en moins de 3 secondes.

1

Montrer l'état actuel dans l'UI

Dans ArgoCD UI : *Applications* → *annuaire-dev*

Montrer que l'app est **Synced ✓** et **Healthy ✓**

Montrer le déploiement avec **1 réplica**

2

Modifier manuellement le cluster (créer le drift)

```
# Dans un terminal – scaler à 3 réplicas "à la main"
kubectl scale deployment annuaire -n devhub-dev --replicas=3
```

Dire à l'oral : "Je viens de modifier le cluster directement, sans passer par Git. C'est ce qu'on appelle un drift."

3

Observer le retour à la normale dans l'UI

Retourner dans ArgoCD UI (rafraîchir si besoin)

L'app passe brièvement à **OutOfSync** puis ArgoCD la resynchronise automatiquement → retour à

Synced

```
# Vérification en CLI (pendant que l'UI affiche)
kubectl get pods -n devhub-dev

# Les 3 pods apparaissent puis 2 sont terminés → retour à 1 seul
```

Dire à l'oral : "ArgoCD a détecté le drift en quelques secondes et a rétabli l'état défini dans Git. Le cluster est self-healing."

4

(Optionnel) Montrer le Pruning

Dans l'UI, montrer l'historique des syncs d'*annuaire-dev* pour voir l'événement de self-healing.



Démonstration du RBAC — Compte developer

⌚ ~1.5 min



Objectif pédagogique : Montrer que les droits ArgoCD sont granulaires — un développeur peut syncer son service sans pouvoir toucher aux autres.

1

Se déconnecter de admin dans l'UI

ArgoCD UI → clic sur l'icône utilisateur en haut à droite → *Log out*

Puis se connecter avec : `developer` / `developerpassword`



Montrer : On peut voir toutes les apps, mais les boutons de configuration sont désactivés

2

Démontrer en CLI : sync autorisé sur annuaire

```
# Login CLI avec le compte developer
argocd login argocd.devhub.local --insecure --username developer --password developerpassword

# Test 1 : Sync autorisé → doit fonctionner
argocd app sync annuaire-dev
→ Attendu : "annuaire-dev" synced successfully ✓
```

3

Démontrer en CLI : sync bloqué sur planning

```
# Test 2 : Sync bloqué → doit retourner une erreur
argocd app sync planning-dev
→ Attendu : FATA[0000] rpc error: code = PermissionDenied ✗
# ArgoCD refuse car le compte developer n'a pas le droit de syncer planning-dev
```

Dire à l'oral : "Le RBAC d'ArgoCD fonctionne de manière très granulaire, au niveau de chaque application. Un développeur frontend ne peut toucher qu'à son service, pas aux autres."

4

Revenir sur le compte admin

```
argocd login argocd.devhub.local --insecure --username admin --password DevHubPassword123!
```



Environnements de Preview Éphémères

⌚ ~1.5
min



Prérequis : La branche `feature-demo-prof` existe déjà en local. L'ApplicationSet est déjà configuré. Il suffit de vérifier que l'app preview est active et montrer l'URL unique.

1

Montrer l'ApplicationSet dans l'UI ArgoCD

Dans ArgoCD UI : *Applications* → `annuaire-preview-feature-demo-prof`

Montrer que c'est une application générée par l'ApplicationSet



Montrer le namespace `devhub-preview-feature-demo-prof` et l'Ingress unique

2

Vérifier que l'application preview est accessible

```
# Vérifier que les pods sont running dans le namespace preview
kubectl get pods -n devhub-preview-feature-demo-prof

# Vérifier l'Ingress
kubectl get ingress -n devhub-preview-feature-demo-prof
→ Attendu : annuaire-feature-demo-prof.devhub.local
```

Ouvrir dans le navigateur : `http://annuaire-feature-demo-prof.devhub.local`

3

Expliquer la création/suppression automatique

Dire à l'oral : "Cette application preview a été créée automatiquement par l'ApplicationSet quand la branche `feature-demo-prof` a été ajoutée à la liste. Si je retire cette branche de la liste et que je commit, ArgoCD va supprimer automatiquement le namespace et toutes ses ressources — c'est le pruning. En production, ça se déclenche à l'ouverture et la fermeture des Pull Requests GitHub."

4

Synthèse finale

Revenir sur la vue principale d'ArgoCD et montrer l'ensemble des applications : `root` → `annuaire-dev`, `planning-dev`, `notif-dev`, `annuaire-preview-feature-demo-prof`

Dire à l'oral : "Tout ce que vous voyez ici est piloté uniquement par des commits dans un dépôt Git. Aucune commande kubectl n'a été utilisée pour créer ou modifier ces ressources en dehors du bootstrap initial. C'est ça, le GitOps."



🔧 CLUSTER & INFRASTRUCTURE

```
# Démarrer le cluster
make cluster-up

# Vérifier les nœuds
kubectl get nodes

# Démarrer le daemon Git local
git daemon --base-path=. \
  --export-all --enable=receive-pack \
  --reuseaddr --port=9000 --verbose &

# Vérifier les pods ArgoCD
kubectl get pods -n argocd
```

🔑 CREDENTIALS

```
# Admin
URL: http://argocd.devhub.local
User: admin
Pass: DevHubPassword123!

# Developer (restreint)
User: developer
Pass: developerpassword
```

🚀 COMMANDES ARGOCD CLI

```
# Login admin
argocd login argocd.devhub.local \
  --insecure --username admin \
  --password DevHubPassword123!

# Login developer
argocd login argocd.devhub.local \
  --insecure --username developer \
  --password developerpassword

# Lister les apps
argocd app list

# Sync app
argocd app sync annuaire-dev
argocd app sync planning-dev # →
PermissionDenied (developer)

# Forcer le refresh
argocd app get annuaire-dev --refresh
```

🌐 URLS LOCALES

http://argocd.devhub.local	← ArgoCD
UI	
http://annuaire.devhub.local	← Annuaire
dev	
http://planning.devhub.local	← Planning
API	
http://annuaire-feature-demo-	
prof.devhub.local	
	← Preview
branch	